



(Self-Improving)² Coding Agents: Curating High-Quality Trajectories via Recursive Self-Improvement

[Repository](#)[Trajectories](#)[Website](#)[Visualization](#)

Abstract

LLM-based coding agents solve long-horizon repository-level tasks by observing the environment and taking actions turn-by-turn, ultimately producing a solution trajectory. In this work, we study how to curate high-quality (both accurate and concise) training trajectories using the code agent’s own backend model. We propose **(Self-Improving)² Coding Agents**, where the model self-improves its own turn-level decisions through candidate branching and selection, with a meta-level optimization framework that recursively discovers better self-improvement strategies. Specifically, we study two strategies for turn-level self-improvement: *Self-Judgement (SJ)* guidance scores each candidate against a set of rubric using the model itself as the judge, and *Self-Likelihood (SL)* guidance scores candidates by their log-probability conditioned on privileged information. On SWE-bench Pro, SJ and SL improve Qwen3.5-122B-A10B’s resolve rate from 48.0% to 58.5% and 53.8%, respectively, with SL requiring less computation. On DeepSWE v1.1, SJ improves GPT-5.6-Terra’s resolve rate from 64.4% to 67.5% and reduces the average number of turns per trajectory from 59.3 to 52.3. Since every-turn branching is computationally expensive, we develop a *recursive self-improvement (RSI)* framework that iteratively discovers more effective and efficient self-improvement strategies by determining when and how to branch. It successfully finds an improved SJ strategy that achieves performance comparable to every-turn counterpart while requiring only 31.5% of its additional compute. The trajectories curated by our self-improvement strategies also provide better training data: a Qwen3.5-35B-A3B-Base student fine-tuned on them resolves 4.2% more of the Python tasks of SWE-bench Pro than one trained on standard rollouts, while producing more concise trajectories.

1. Introduction

Training capable LLM-based coding agents requires repository-level tasks paired with executable environments and solution trajectories. A line of research (Yang et al., 2025; Badertdinov et al., 2025, 2026) has substantially scaled the task side by constructing problems together with their corresponding environments. However, high-quality solution trajectories remain scarce despite serving as a primary source of supervision for training coding agents. Even successful rollouts sampled directly from the policy often contain intermediate failures, ineffective actions, or excessive trial and error (Chen et al., 2025; Wang et al., 2025). To bridge this gap, we study how to curate *high-quality (accurate and concise)* coding-agent trajectories using the agent’s own backend model, with each step being grounded in prior observations, advancing toward the target solution, and contributing new information rather than repeating what has already been established, which could serve as training data that help coding agents solve long-horizon coding tasks efficiently and effectively.

Existing work has shown that LLMs can improve their initial responses through self-critique and revision (Madaan et al., 2023; Jiang et al., 2025), and has identified that models can evaluate concrete candidates more reliably than they can generate correct responses (Song et al., 2025). Coding-agent trajectories provide a natural

setting in which to exploit this capability at a finer granularity: because a trajectory unfolds through successive action–observation turns, poor decisions can be filtered before they influence subsequent turns (Yang et al., 2024). This motivates turn-level self-improvement, where bad behaviors are filtered at the turns where they arise rather than after the entire trajectory has been generated.

We propose **(Self-Improving)² Coding Agents**, where a model self-improves its trajectories during generation through two turn-level strategies, *Self-Judgement (SJ)* and *Self-Likelihood (SL)*, while a meta-level *recursive self-improvement (RSI)* framework further improves the strategies themselves. For both strategies, the model samples a set of candidate actions, self-evaluates them, and executes only the preferred one. SJ scores each candidate against rubrics on the soundness of its reasoning and action, probably also with the alignment with additional privileged information (PI), while SL evaluates candidates by the log-probability the model assigns to them once PI is placed in context. Empirically, the former one brings greater quality improvements, while the latter is a lower-cost alternative with more modest performance improvements. *The PI for evaluation in both strategies never enters the trajectory generation context*, ensuring that the curated trajectories retain the same form as standard turn-by-turn rollouts and can be used directly for distillation or RL training.

However, applying self-improvement at every turn is computationally expensive: many turns are effectively deterministic, and we observe that a substantial fraction ($\sim 18\%$) of branched candidates are functionally equivalent. To improve the efficiency of both strategies, we introduce a recursive self-improvement framework that treats the self-improvement strategy itself as an optimization problem, using a Proposer–Executor–Recorder loop to iteratively discover more efficient and effective settings. In each cycle, the Proposer suggests when and how much to branch, an Executor evaluates it by trajectory generation, and the Recorder summarizes the outcome for the next cycle, where the Proposer and Recorder are implemented with frontier models, e.g., Claude Opus 5 (Anthropic, 2026) and GPT-5.6-Sol (OpenAI, 2026), while the Executor is implemented with the open-source policy model. The ultimate goal is to identify a single generalizable strategy that enables efficient trajectory generation with substantially reduced computational overhead. Taken together, our framework performs self-improvement at two levels: turn-level action improvement and meta-level strategy optimization.

Our experiments show that both turn-level self-improvement strategies produce accurate and concise trajectories, while the strategy discovered by the RSI framework preserves these benefits at substantially lower cost. On SWE-bench Pro (Deng et al., 2025), SJ increases the resolve rate of Qwen3.5-122B-A10B (Qwen Team, 2026) from 48.0% to 58.5%. This improvement transfers to frontier models: on DeepSWE v1.1 (Huang et al., 2026), SJ raises the resolve rate of GPT-5.6-Terra from 64.4% to 67.5%, while reducing the average trajectory turns from 59.3 to 52.3. Moreover, the RSI framework discovers an *early-commitment window* strategy, branching from the first observation until the third file-mutating command, which matches every-turn SJ while using only 31.5% of its additional compute. Most importantly, the trajectories curated by our strategies make better training data: fine-tuned on every-turn SJ trajectories, Qwen3.5-35B-A3B-Base resolves 4.2% more tasks on the Python subset of SWE-bench Pro than the standard counterpart and produces more concise trajectories (86.6 vs. 93.8 mean turns), and the discovered strategy matches this gain at a lower curation cost.

Our contributions are threefold: (1) We formulate coding-agent trajectory curation as turn-level self-improvement and propose two strategies, *Self-Judgement (SJ)* and *Self-Likelihood (SL)*, for generating more accurate and more concise trajectories. (2) Taking a step further, we introduce a meta-level recursive self-improvement framework that iteratively optimizes the self-improvement strategy itself, enabling the discovery of generalizable strategies that substantially reduce computational cost. (3) We show that this two-level framework consistently improves trajectory accuracy and conciseness across models and benchmarks, while the searched strategy achieves performance comparable to every-turn self-improvement with less additional compute.

2. Related Work

2.1. Data Curation for Coding Agents

Recent efforts in data curation for coding agents include mining executable environments from real-world repositories at scale (SWE-Rebench and SWE-Rebench-V2 (Badertdinov et al., 2025, 2026), SWEFlow (Zhang et al., 2025)), generating synthetic tasks from existing codebases (SWE-Smith (Yang et al., 2025), R2E-Gym (Jain et al., 2025)), synthesizing problems or subproblems tailored to models’ capabilities and weaknesses (Liang

et al., 2025, 2026a,b), and bootstrapping stronger agents from zero-shot rollouts (SWE-ZERO and SWE-HERO (Ludwig et al., 2026)). For trajectory optimization, Ma et al. (2026); Han et al. (2026) use process-level supervision, such as process graphs or external reward models, to guide trajectory generation, whereas Wang et al. (2026a); Zheng et al. (2026) remove erroneous and redundant steps through post-processing. In our work, we study how LLM-based agents can generate accurate, concise trajectories using their own backend model through turn-level self-improvement, proposing and refining their own trajectories without relying on external teachers.

2.2. Self-Improving Agents

Existing self-improvement for LLM agents has mostly taken the form of post-hoc reflection: Self-Refine (Madaan et al., 2023) feeds the model’s own critique back into its context, while Gödel Agent (Yin et al., 2025) and SICA (Robeyns et al., 2025) let the agent rewrite its own prompts or source code. WebClipper (Wang et al., 2026a) prunes redundant steps from web-agent trajectories via a graph-based post-processing pass. Tree of Thoughts (Yao et al., 2023) instead branches over intermediate steps and selects with an explicit value function, and Jali et al. (2026) varies the compute spent per turn of an agentic trajectory. In comparison, our work not only develops self-improvement strategies for problem solving, but also introduces a recursive self-improvement framework that iteratively improves the effectiveness and efficiency of strategies themselves.

2.3. Recursive Self-Improvement

Recent advances in LLM agentic systems have enabled autonomous research (Lu et al., 2024; Ghareeb et al., 2026) in the style of recursive self-improvement. XYZ Agentic Team (2026) presents an AI4AI system in which AI agents participate across the full model-development loop, including task construction, training, runtime optimization, and evaluation. AREX (Lu et al., 2026) recursively improves deep research through verification and refinement, and BigBang (BIGBANG team, 2026) self-evolves frontier tasks through iterative generation and verification. A parallel thread optimizes the agentic harness around a fixed model: Meta-Harness (Lee et al., 2026b), Recursive Harness Self-Improvement (Lee et al., 2026a) and Agentic Harness Engineering (Lin et al., 2026a) evolve prompts, tools and control flow end to end. Two recent studies caution that such loops can be misleading (Lin et al., 2026b; Wang et al., 2026b), showing that harness updates do not necessarily yield genuine, generalizable improvements, while gains may arise from sampling randomness and overfitting. To ensure practical and reliable discovery, our framework explicitly constrains the operation space of the RSI framework, enforces strict anti-hacking rules, and evaluates promising discovered strategies on a broader set of *held-out* benchmark instances to assess their generalizability.

3. Method

3.1. Task Formulation

A coding task x consists of a problem statement, a code repository, and an associated Docker environment. An agent built on a language model π_θ solves it over a sequence of turns: at turn t it emits an action a_t (a reasoning trace followed by a command), the environment returns an observation $o_t = \text{Env}(a_t)$, and the pair is appended to the trajectory prefix $\tau_{<t} = (a_1, o_1, \dots, a_{t-1}, o_{t-1})$. Standard generation samples $a_t \sim \pi_\theta(\cdot \mid x, \tau_{<t})$ until the agent submits a patch or a turn budget T is exhausted. In our setting, we assume access to *privileged information* (PI) z about the task for stronger guidance, namely the reference solution, or a natural-language hint (Appendix A). Notably, z never enters the policy’s context: every action in τ is sampled from $\pi_\theta(\cdot \mid x, \tau_{<t})$, ensuring that τ remains a valid training trajectory for a model that will not have access to z at test time.

Our method comprises two turn-level self-improvement strategies, *Self-Judgement* (SJ) and *Self-Likelihood* (SL), which share the same skeleton (Algorithm 1). At each turn the policy proposes k candidates,

$$a_t^{(1)}, \dots, a_t^{(k)} \stackrel{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x, \tau_{<t}), \quad (1)$$

which are then scored by the same model. Only the highest-scoring candidate is executed. The two strategies differ only in their selection mechanisms (Figure 1).

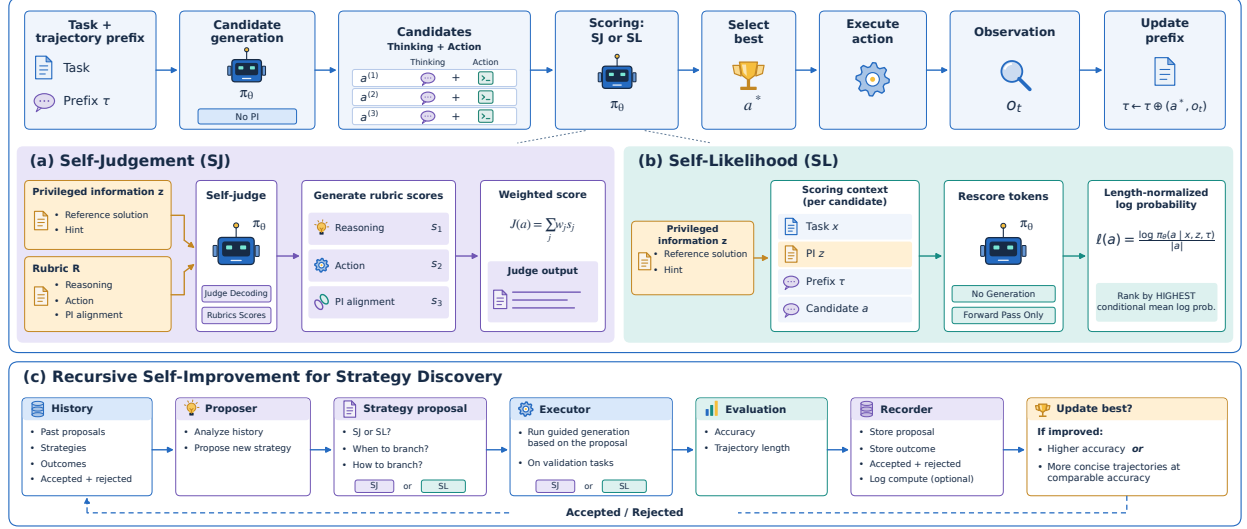


Figure 1: Overview of our method for curating high-quality (accurate and concise) coding agent trajectories, which includes two turn-level self-improvement strategies, Self-Judgement (SJ) and Self-Likelihood (SL), and a meta-level recursive self-improvement framework that iteratively optimizes these strategies.

3.2. Self-Judgement Guidance

Self-judgement guidance turns the model into its own judge. Given a candidate $a_t^{(i)}$, the same model π_θ is prompted to score it against a fixed rubric $\mathcal{R} = \{r_1, \dots, r_m\}$ with weights $w_j \geq 0$, $\sum_j w_j = 1$. The rubric items assess the quality of the reasoning, the soundness of the proposed action, and optionally the candidate’s alignment with the PI, which is accessible only to the self-judge and enables more informed candidate selection. The self-judge prompt P_{judge} (Appendix E.2) conditions on the problem statement, the trajectory prefix, the candidate, the privileged information z and the rubric $\mathcal{R}$, and returns a vector of scores:

$$s^{(i)} = (s_1^{(i)}, \dots, s_m^{(i)}) \sim \pi_\theta(\cdot \mid P_{\text{judge}}(x, \tau_{<t}, a_t^{(i)}, z, \mathcal{R})), \quad s_j^{(i)} \in [0, 1], \quad (2)$$

where each item is scored on a bounded ordinal scale that we normalize to $[0, 1]$. The rubric score of a candidate is the weighted sum of its item scores, and the candidate with the highest score is selected:

$$J(a_t^{(i)}) = \sum_{j=1}^m w_j s_j^{(i)}, \quad i^* = \arg \max_{i \in [k]} J(a_t^{(i)}). \quad (3)$$

The executed command is $a_t = a_t^{(i^*)}$. When multiple candidates propose identical commands, only one candidate is randomly selected from each group of duplicates for self-judging.

3.3. Self-Likelihood Guidance

Self-judgement incurs the additional cost of generating judge completions for all k candidates. To score candidates more efficiently, we propose self-likelihood guidance, which replaces the generative judge with a likelihood test that generates no tokens. The intuition is that once the model is given access to the PI, it should assign the highest confidence to the candidate that is most likely to be heading in the right direction. We therefore score each candidate by the length-normalized conditional log-likelihood that the same model assigns to it once the PI has been inserted into the context.

$$\ell(a_t^{(i)}) = \frac{1}{|a_t^{(i)}|} \sum_{u=1}^{|a_t^{(i)}|} \log \pi_\theta(a_{t,u}^{(i)} \mid x, z, \tau_{<t}, a_{t,<u}^{(i)}), \quad i^* = \arg \max_{i \in [k]} \ell(a_t^{(i)}), \quad (4)$$

Algorithm 1 Trajectory generation with self-judgement and self-likelihood guidance.

Require: task x ; policy π_θ ; privileged information z ; candidate factor k ; selector $s \in \{\text{SJ}, \text{SL}\}$

```
1:  $\tau \leftarrow \emptyset$ 
2: for  $t = 1, 2, \dots$  do
3:   sample  $a^{(i)} \sim \pi_\theta(\cdot \mid x, \tau)$  for  $i = 1, \dots, k$   $\triangleright$  candidates from the unprivileged policy
4:   if all  $a^{(i)}$  propose the same command then
5:      $i^* \leftarrow 1$ 
6:   else if  $s = \text{SJ}$  then
7:      $i^* \leftarrow \arg \max_i J^{(i)}, J^{(i)} \sim \pi_\theta(\cdot \mid P_{\text{judge}}(x, \tau, a^{(i)}, z, \mathcal{R}))$  for  $i = 1, \dots, k$   $\triangleright$  self-judgement
8:   else
9:      $i^* \leftarrow \arg \max_i \frac{1}{|a^{(i)}|} \log \pi_\theta(a^{(i)} \mid x, z, \tau)$  for  $i = 1, \dots, k$   $\triangleright$  self-likelihood
10:  end if
11:   $a_t \leftarrow a^{(i^*)}; o_t \leftarrow \text{Env}(a_t); \tau \leftarrow \tau \oplus (a_t, o_t)$   $\triangleright$  execute only the selected candidate
12:  if  $a_t$  is a submission then
13:    break
14:  end if
15: end for
16: return  $\tau$ 
```

where $a_{t,u}^{(i)}$ denotes the u -th token of the candidate. Equivalently, the candidate with the lowest per-token negative log-likelihood under the privileged context wins. Subtracting the unprivileged log-likelihood from Equation (4) turns the score into the pointwise mutual information between the candidate and z , which measures how much z specifically explains the candidate and can likewise be used to rank candidates. As the empirical results in Appendix C.3 show, the conditional likelihood alone performs better, so we use it in our main experiments. Note that computing ℓ requires only a single forward pass per candidate and no decoding, and the k passes share the prefix, so the selector adds negligible cost on top of generating the candidates. We ablate both the placement of z in the context and the form in which z is provided in Section 5.2.

3.4. Recursive Self-Improvement for Strategy Optimization

Both strategies above branch at *every* turn, incurring unnecessary computation when branching and selection are unlikely to affect the trajectory. A more efficient approach is to selectively apply self-improvement by determining when and how to branch. To this end, we introduce a recursive self-improvement framework to discover more efficient strategies while maintaining task performance, which runs in cycles over three agents: a **Proposer** $\mathcal{P}$ writes one candidate strategy from the memory $\mathcal{M}_t$ of earlier cycles, an **Executor** π_θ generates trajectories with it on a validation set $\mathcal{D}_{\text{val}}$, and a **Recorder** $\mathcal{W}$ summarizes and appends the results into $\mathcal{M}_t$. A proposed strategy is accepted only if it improves upon the current best strategy on the validation split, either by solving more problems or by producing more concise trajectories at comparable accuracy.

Search space. A strategy $\sigma = (\beta, s) \in \Sigma$ has two components: a *branching rule* $\beta(x, \tau_{<t}) \in \{1, 2, \dots\}$ decides *where* to branch and *how many* candidates to generate, and a *selector* $s \in \{\text{SJ}, \text{SL}\}$ decides which method to optimize. The strategy is also kept from modifying the policy’s context during trajectory generation, ensuring the generated trajectories remain suitable as training data. Moreover, β is prohibited from conditioning task identity; instead, it typically relies on behavioral signals available at each turn, such as exit codes, whether an edit has been made, or repeated commands. This restriction prevents the searched strategy from memorizing validation tasks (Lee et al., 2026b).

The search loop. Each cycle begins with a proposal. At cycle t , the search state consists of the incumbent strategy σ_t and the search history $\mathcal{M}_t$, which records all strategies explored in previous cycles. $\mathcal{P}$ reviews $\mathcal{M}_t$,

together with a sample of anonymized trajectories $\tilde{\mathcal{T}}_t$ generated using σ_t , and proposes a strategy $\tilde{\sigma}_t$:

$$\tilde{\sigma}_t = \mathcal{P}(\sigma_t, \mathcal{M}_t, \tilde{\mathcal{T}}_t), \quad \tilde{\sigma}_t \in \Sigma. \quad (5)$$

The strategy $\tilde{\sigma}_t$ is then adopted by the Executor π_θ to generate trajectories on the validation set $\mathcal{D}_{\text{val}}$. The Proposer $\mathcal{P}$ has no access to these tasks and must therefore propose strategies that generalize across tasks. The resulting trajectories are then evaluated, yielding the evidence:

$$e_t = \mathcal{E}(\pi_\theta, \tilde{\sigma}_t, \mathcal{D}_{\text{val}}) = (A(\tilde{\sigma}_t), C(\tilde{\sigma}_t), \Phi(\tilde{\sigma}_t), \mathcal{T}_t), \quad (6)$$

where $\mathcal{E}$ denotes the evaluation procedure, A is the number of resolved tasks, C measures retained-trajectory length in generated tokens and turns, with lower values indicating greater conciseness, Φ the realized branching rate and decision breakdown, and $\mathcal{T}_t$ the trajectories. These length measures exclude the additional computation used for branching, candidate selection, and other self-improvement operations; they measure trajectory conciseness rather than total generation cost. Although Φ is recorded for analysis, branching frequency is not an acceptance criterion. The proposed strategy $\tilde{\sigma}_t$ replaces the incumbent if it satisfies the acceptance criteria:

$$\sigma_{t+1} = \begin{cases} \tilde{\sigma}_t & \Delta_t \geq \delta, \\ \tilde{\sigma}_t & |\Delta_t| < \delta \text{ and } \min(\rho_t^{\text{tok}}, \rho_t^{\text{turn}}) \leq \rho_{\text{max}} \text{ and } \Delta_t \geq -\epsilon, \\ \sigma_t & \text{otherwise.} \end{cases} \quad (7)$$

$\Delta_t = A(\tilde{\sigma}_t) - A(\sigma_t)$ denotes the paired change in solved count, while ρ_t^{tok} and ρ_t^{turn} denote the $\tilde{\sigma}_t$ -to- σ_t ratios of trajectory token counts and turn counts, respectively. The first condition captures an accuracy gain, whereas the second captures improved trajectory conciseness with negligible loss in accuracy.

At the end of each cycle, the Recorder $\mathcal{W}$ reviews $\tilde{\sigma}_t$, its evaluation results and generated trajectories in e_t , and the updated incumbent σ_{t+1} , then appends a summary m_t to the persistent search history:

$$m_t = \mathcal{W}(\tilde{\sigma}_t, e_t, \sigma_{t+1}), \quad \mathcal{M}_{t+1} = \mathcal{M}_t \oplus m_t. \quad (8)$$

Here, $\oplus$ denotes appending a record without modifying earlier entries. Both accepted and rejected candidates are retained so that subsequent proposals can build on previous successes and failures. The next cycle begins from $(\sigma_{t+1}, \mathcal{M}_{t+1})$. The loop is recursive but bounded: the objective, admissible search space, validation split, and acceptance rule are fixed in advance; only the strategy is searched.

4. Experiments

4.1. Setup

Models and Benchmarks. We primarily run experiments on two open-weight models, [Qwen3.5-35B-A3B](#) and [Qwen3.5-122B-A10B](#) ([Qwen Team, 2026](#)). To evaluate the scalability of our method, we additionally conduct experiments with two frontier models, [GPT-5.6-Terra](#) and [GPT-5.6-Luna](#). We evaluate on [SWE-bench Verified](#) (500 tasks) ([Chowdhury et al., 2024](#)) and [SWE-bench Pro](#) ([Deng et al., 2025](#)) (731 tasks); For experiments beyond Table 1, we use the 266-task Python subset of SWE-bench Pro for computational feasibility. The GPT-5.6 models are evaluated on [DeepSWE v1.1](#) ([Huang et al., 2026](#)), a benchmark of 113 more challenging SWE tasks. We use the resolve rate to assess the trajectory quality. For efficiency measurement, we report the mean, median, and 90th percentile of the number of turns per trajectory.

Implementations. We use the mini-SWE-agent ([Yang et al., 2024](#)) harness, providing the model with access only to Bash commands. Trajectories are capped at 250 turns for SWE-bench and 1,000 for DeepSWE. Decoding uses temperature 1.0, top- p 0.95, top- k 10 and at most 4,096 new tokens per turn for all experiments.

Table 1: Main results on SWE-bench Verified and SWE-bench Pro. Best value per model in **bold**; parentheses give the change against the standard baseline of the same model (green = improvement, red = regression).

Model	Strategy	Resolve rate (%)		Turns – Verified			Turns – Pro		
		Verified	Pro	Mean	Med.	P90	Mean	Med.	P90
Qwen3.5-35B-A3B	Standard	65.8	46.0	85.0	76	144	89.4	80	153
	Self-judgement	70.0 (↑4.2)	53.2 (↑7.2)	73.7 (↓11.3)	67 (↓9)	127 (↓17)	79.3 (↓10.1)	72 (↓8)	133 (↓20)
	Self-likelihood	66.8 (↑1.0)	51.2 (↑5.2)	76.9 (↓8.1)	68 (↓8)	130 (↓14)	77.5 (↓11.9)	68 (↓12)	134 (↓19)
Qwen3.5-122B-A10B	Standard	67.0	48.0	72.1	64	122	84.6	80	135
	Self-judgement	71.0 (↑4.0)	58.5 (↑10.5)	68.8 (↓3.3)	61 (↓3)	116 (↓6)	80.2 (↓4.4)	74 (↓6)	130 (↓5)
	Self-likelihood	69.8 (↑2.8)	53.8 (↑5.8)	69.5 (↓2.6)	62 (↓2)	118 (↓4)	69.9 (↓14.7)	64 (↓16)	117 (↓18)

Table 2: Frontier GPT-5.6 models on DeepSWE v1.1 (113 tasks). Only the GPT-5.6-Terra rows are averaged over four independent runs of the full benchmark; the GPT-5.6-Luna rows come from a single run.

Model	Strategy	Resolve rate (%)	Turns		
			Mean	Med.	P90
GPT-5.6-Luna-Max	Standard	60.2	246.8	173	465
	Self-judgement	64.6 (↑4.4)	223.0 (↓23.8)	150 (↓23)	474 (↑9)
GPT-5.6-Terra-xHigh	Standard	64.4 ± 2.0	59.3	52	98
	Self-judgement	67.5 ± 2.5 (↑3.1)	52.3 (↓7.0)	46 (↓6)	86 (↓12)

For *self-judgement guidance* (SJ), the policy samples $k = 2$ candidate turns at each step and scores each by averaging $n = 3$ judgements. Alignment with privileged information receives a rubric weight of 0.30, with the remaining 0.70 distributed among the other criteria. For *self-likelihood guidance* (SL), the policy samples $k = 4$ candidate turns at each step and selects the candidate with the highest log-probability conditioned on privileged information. SJ uses the gold patch as privileged information, whereas SL uses a hint derived from the gold patch and problem statement. Detailed prompts for both methods are provided in Appendix E.

RSI framework. The **Proposer** and **Recorder** are agents implemented through GitHub Copilot CLI, iteratively backed by either **Claude Opus 5** or **GPT-5.6-Sol** and each with a dedicated skill document. We serve Qwen3.5-122B-A10B with SGLang (Zheng et al., 2024) as the **Executor**, which generates trajectories according to the proposed strategy. Three baseline strategies are initially provided to the Proposer: standard, every-turn SJ, and every-turn SL. Each subsequent proposal focuses on optimizing either SJ or SL. The validation split $\mathcal{D}_{\text{val}}$ consists of 192 problems sampled from **SWE-bench Multilingual**. We set the promotion threshold at $\delta = 7$ solved tasks, $\rho_{\text{max}} = 0.9$ and $\epsilon = 2$ in Equation (7).

4.2. Main Results

Qwen3.5 on SWE-bench. Table 1 presents results for both strategies on SWE-bench Verified and SWE-bench Pro. Self-judgement improves resolve rates on both benchmarks, with the largest gain on the more challenging Pro benchmark, where Qwen3.5-122B-A10B improves from 48.0% to 58.5%. It also shortens trajectories: on Verified, the mean number of turns for Qwen3.5-35B-A3B decreases from 85.0 to 73.7. The less computationally demanding self-likelihood guidance also improves the resolve rate on Pro, achieving 5.8% improvement for the 122B model, while reducing its mean turn count from 84.6 to 69.9.

GPT-5.6 on DeepSWE. Table 2 compares standard generation with self-judgement¹ for GPT-5.6-Terra at the xHigh reasoning effort and GPT-5.6-Luna at the max reasoning effort on the DeepSWE v1.1 benchmark, with

¹The API does not expose token-level probabilities, preventing us from evaluating the self-likelihood setting.

Table 3: Comparison of standard generation, every-turn self-judgement guidance, and the efficient self-judgement strategy discovered through the RSI strategy optimization system on Qwen3.5-122B-A10B.

Benchmark	Strategy	Resolve rate (%)	Turns			Branching cost	
			Mean	Med.	P90	Extra cand.	Extra tokens
SWE-bench Verified	Standard	67.0	72.1	64	122	–	–
	Full self-judgement	71.0 ($\uparrow 4.0$)	68.8 ($\downarrow 3.3$)	61 ($\downarrow 3$)	116 ($\downarrow 6$)	34,393	7.8M
	Discovered strategy	71.2 ($\uparrow 4.2$)	71.9 ($\downarrow 0.2$)	65 ($\uparrow 1$)	120 ($\downarrow 2$)	9,633 ($\downarrow 72.0\%$)	2.1M ($\downarrow 73.1\%$)
SWE-bench Pro (Python)	Standard	55.3	78.9	74	126	–	–
	Full self-judgement	65.4 ($\uparrow 10.1$)	77.6 ($\downarrow 1.3$)	71 ($\downarrow 3$)	124 ($\downarrow 2$)	20,642	4.7M
	Discovered strategy	60.6 ($\uparrow 5.3$)	73.3 ($\downarrow 5.6$)	67 ($\downarrow 7$)	124 ($\downarrow 2$)	7,728 ($\downarrow 62.6\%$)	1.8M ($\downarrow 61.7\%$)

Table 4: Evaluation of data-curation strategies through supervised fine-tuning into Qwen3.5-35B-A3B-Base. The resulting models are evaluated on SWE-bench Verified and the Python subset of SWE-bench Pro.

Student	Resolve rate (%)		Turns – Verified			Turns – Pro (Python)		
	Verified	Pro (Python)	Mean	Med.	P90	Mean	Med.	P90
Qwen3.5-35B-A3B-Base	24.0	12.4	33.7	28	64	41.6	32	79
↳ SFT on Standard	57.4	46.2	91.1	76	172	93.8	80	179
↳ SFT on Full self-judgement	59.6 ($\uparrow 2.2$)	50.4 ($\uparrow 4.2$)	83.4 ($\downarrow 7.7$)	72 ($\downarrow 4$)	140 ($\downarrow 32$)	86.6 ($\downarrow 7.2$)	75 ($\downarrow 5$)	159 ($\downarrow 20$)
↳ SFT on Discovered strategy	58.6 ($\uparrow 1.2$)	50.4 ($\uparrow 4.2$)	95.8 ($\uparrow 4.7$)	82 ($\uparrow 6$)	175 ($\uparrow 3$)	92.8 ($\downarrow 1.0$)	79 ($\downarrow 1$)	170 ($\downarrow 9$)

the Terra results averaged over four runs and the Luna results from a single run. Self-judgement increases the resolve rate of Terra from 64.4% to 67.5% while producing more concise trajectories, reducing the mean number of turns from 59.3 to 52.3. The effect holds for Luna, whose resolve rate rises from 60.2% to 64.6% with the mean number of turns falling from 246.8 to 223.0. These results provide further evidence that self-judgement improves both accuracy and trajectory conciseness across model capabilities and benchmarks.

4.3. Recursive Self-Improvement Framework

We ran the RSI strategy optimization framework in Section 3.4 for 15 cycles. Most proposals were rejected; several solved 6 more tasks than the incumbent, yet fell short of the acceptance threshold of $\delta = 7$. The strategy accepted at cycle 6 remained the best after all 15 cycles. It uses an *early-commitment window*: *branch with $k = 2$ from the first observation until the third file-mutating command has executed, then follow a single path*. Its premise is that early decisions determine a trajectory, while the model can proceed effectively on its own after the third edit. We therefore selected this strategy to evaluate its generalizability and the training utility of its curated trajectories. Detailed records of all RSI optimization cycles are provided in Appendix D.

Table 3 evaluates this discovered strategy on all SWE-bench Verified and Pro (Python) tasks, none of which the Proposer ever observed during the recursive optimization. On Verified, the strategy matches full self-judgement (71.2% vs. 71.0%), while branching at only 28% of turns and generating 9,633 additional candidates instead of 34,393. On Pro (Python), it retains a clear gain over the standard baseline, raising the resolve rate from 55.3% to 60.6% at roughly one third the branching cost of every-turn self-judgement, while also producing more concise trajectories (mean turns 73.3 vs. 77.6). Together, these results show that a substantially more efficient self-improvement strategy exists and can be discovered by our RSI framework, and that, for coding-agent tasks, concentrating self-improvement on the early turns may benefit both accuracy and efficiency.

4.4. Curating Data for Distillation

The motivation for our self-improvement methods is to curate high-quality trajectories as training data. We validate this by fine-tuning the pre-trained checkpoint *Qwen3.5-35B-A3B-Base* on trajectories generated by Qwen3.5-122B-A10B under three settings: *standard* sampling, *every-turn self-judgement*, and the more efficient self-judgement strategy discovered in section 4.3. The trajectories are generated on a pool of about 10,780 Python SWE tasks with executable environments and reference patches, drawn from SWE-smith (Yang et al.,

Table 5: Evaluating the effectiveness of providing privileged information (PI) for self-judgement. In the w/ PI setting, the judge additionally considers each candidate’s alignment with the PI (gold patch in our experiments); in the w/o PI setting, candidates are scored solely using the rubric for reasoning and actions.

Model	Strategy	Resolve rate (%)		Turns – Verified			Turns – Pro (Python)		
		Verified	Pro (Python)	Mean	Med.	P90	Mean	Med.	P90
Qwen3.5-35B-A3B	Standard	65.8	50.4	85.0	76	144	92.0	84	150
	Self-judgement w/o PI	67.2 (↑1.4)	50.4 (0.0)	80.0 (↓5.0)	72 (↓4)	136 (↓8)	86.8 (↓5.2)	82 (↓2)	140 (↓10)
	Self-judgement w/ PI	70.0 (↑4.2)	56.0 (↑5.6)	73.7 (↓11.3)	67 (↓9)	127 (↓17)	75.4 (↓16.6)	69 (↓15)	120 (↓30)

Table 6: Ablation on Qwen3.5-122B-A10B for self-likelihood guidance: *what* privileged information (PI) to condition on (the gold patch vs. a hint) and *where* to place it (*before*: between the problem statement and the prefix trajectory; *after*: after the prefix trajectory).

PI	Placement	Resolve rate (%)		Turns – Verified			Turns – Pro (Python)		
		Verified	Pro (Python)	Mean	Med.	P90	Mean	Med.	P90
Standard		67.0	55.3	72.1	64	122	78.9	74	126
Gold patch	Before	67.8 (↑0.8)	56.0 (↑0.7)	71.5 (↓0.6)	62 (↓2)	129 (↑7)	66.0 (↓12.9)	61 (↓13)	110 (↓16)
	After	69.4 (↑2.4)	57.5 (↑2.2)	64.6 (↓7.5)	60 (↓4)	108 (↓14)	62.8 (↓16.1)	58 (↓16)	106 (↓20)
Hint	Before	69.8 (↑2.8)	60.5 (↑5.2)	69.5 (↓2.6)	62 (↓2)	118 (↓4)	65.3 (↓13.6)	60 (↓14)	114 (↓12)
	After	68.8 (↑1.8)	59.4 (↑4.1)	65.3 (↓6.8)	59 (↓5)	110 (↓12)	66.1 (↓12.8)	61 (↓13)	108 (↓18)

2025) and SWE-rebench series (Badertdinov et al., 2025, 2026). All students are trained with the same recipe and evaluated under the settings of Table 1; the full data-curation and fine-tuning settings are in Appendix B.

Table 4 reports the results. Trajectories curated by self-judgement make better training data than standard rollouts: the student trained on them resolves 2.2% more tasks on Verified and 4.2% more on Pro (Python), and its trajectories are also more concise, with the mean number of turns falling from 91.1 to 83.4 on Verified and from 93.8 to 86.6 on Pro. The discovered strategy setting retains most of the accuracy gains (+1.2% on Verified and +4.2% on Pro), although its trajectories are less concise than those produced by every-turn self-judgement. This is expected, as the discovered strategy performs trajectory refinement only in the early-commitment window, which may leave some later redundant turns unpruned.

5. Ablation Studies

We ablate two design choices: the effect of privileged information (PI) in self-judgement, and what the self-likelihood score is conditioned on and where the PI is inserted.

5.1. The Effect of Privileged Information in Self-Judgement

Table 5 ablates the effect of whether the self-judge is shown the PI, namely the gold patch in our experiments. Self-judgement without PI already helps: guided only by rubrics on reasoning and actions, it raises the Verified resolve rate from 65.8% to 67.2% and condenses trajectories. Adding PI to the judge’s context and evaluating the candidate turn’s alignment with it further extends the gain on Verified (+4.2 vs. +1.4 points), and improves Pro (Python) by 5.6%. It also further improves trajectory conciseness, reducing the average number of turns from 80.0 to 73.7 on Verified and from 86.8 to 75.4 on Pro compared with the w/o PI setting.

5.2. What Should Self-Likelihood Guidance Condition on, and Where?

Table 6 ablates two choices for self-likelihood guidance on Qwen3.5-122B-A10B: *what* PI the likelihood is conditioned on, the gold patch or a natural-language hint (check Appendix A), and *where* the PI is placed, before the prefix trajectory or after it. Generally, every configuration improves over standard generation

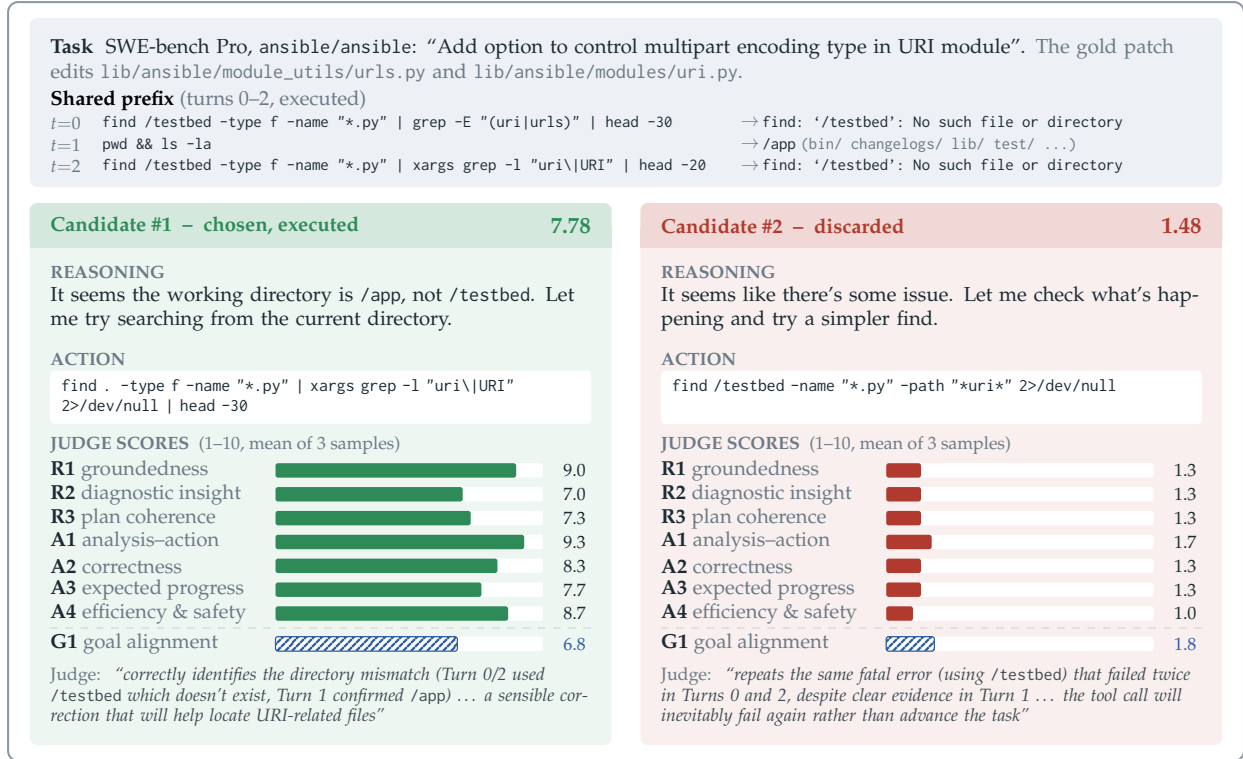


Figure 2: An example of candidate branching and selection under self-judgement on Qwen3.5-35B-A3B.

on both resolve rate and trajectory conciseness. The hint is the better PI for accuracy: placed before the trajectory it gives the best resolve rates on Verified (69.8%) and Pro (Python) (60.5%), a 5.2-point gain on Pro against 2.2 for the gold-patch configuration. One potential reason is that the hint is more concise (around 1k characters) and consistently formatted, making it easier for the model to utilize effectively in the context-aware self-likelihood guidance. Placement has a smaller effect on accuracy than on conciseness: inserting the PI after the trajectory produces the shortest trajectories, with the gold patch reducing the mean number of turns on Pro from 78.9 to 62.8. However, placing the PI after the trajectory may have a larger effect on the likelihood of the candidate turn as it disrupts the continuity of the trajectory, potentially introducing noise and instability in the model’s predictions. Based on the observations above, we adopt *hint, before* as the default configuration in the main experiments, as it achieves the highest accuracy while still condensing trajectories by 13.6 turns on Pro.

6. Analysis

6.1. Case Study: Self-Judgement Rejects Repeated Mistakes

Figure 2 shows a representative instance of self-improvement in a SWE-bench Pro trajectory generated by Qwen3.5-35B-A3B, where two candidates share the same trajectory prefix but diverge in both reasoning and action. The chosen candidate has *understood the last observation*: it notes that the working directory is /app, drops the wrong path, and searches file contents for the identifier the task depends on. By contrast, the discarded candidate searches the non-existent /testbed again. The self-judgement correctly identifies this repeated mistake, assigning the chosen candidate substantially higher scores across both reasoning and action criteria, as well as a higher alignment score with the PI. This example illustrates that the turn-level self-improvement can identify undesirable patterns in the policy’s own generations. Filtering out such cases leads to both more accurate and more concise trajectories than standard turn-by-turn generation.

7. Conclusion

We presented **(Self-Improving)² Coding Agents**, a framework for curating high-quality (accurate and concise) coding-agent trajectories using the agent’s own backend model. Specifically, we study two turn-level self-improvement strategies: *Self-Judgement* scores candidates against rubrics on the soundness of their reasoning and actions, while *Self-Likelihood* guidance selects candidates by the log-probability assigned by the model when privileged information is provided in context. At the meta level, we introduce a recursive self-improvement framework that treats the self-improvement strategy itself as an optimization problem, discovering when and how to apply turn-level self-improvement effectively while reducing computational cost and maintaining performance comparable to every-turn branching. Across open-weight and frontier models on SWE-bench Verified, SWE-bench Pro, and DeepSWE, we demonstrate the effectiveness of our whole framework in raising resolve rates and condensing trajectories, and the trajectories it curates make better training data for distillation.

Limitations. Both of our self-improvement strategies could potentially discard a flawed action at the turn where it would have been taken. In standard trajectories, by contrast, the model may make a mistake early and repairs it several turns later. Our curated trajectories therefore contain fewer of these error-recovery episodes, so a student trained on them alone may acquire less of the ability to notice and fix its own earlier mistakes. A natural remedy is to mix the curated trajectories with standard ones that retain a mistake together with its later correction, which we leave to future work.

Future Work. We identify three directions for future work. First, our data curation strategies can also supply the policy with better positive samples for agentic reinforcement learning (RL), which should raise the upper bound of the policy’s problem-solving ability beyond standard agentic RL on its own rollouts, which we plan to validate. Second, our experiments cover coding agents only, but both strategies require nothing more than a turn-structured trajectory and a harness that can draw candidate actions, so they transfer to other agentic tasks as is. Third, it remains open whether a dedicated judge or likelihood model, post-trained from the policy, would provide a stronger signal for evaluating actions and thereby guide the policy toward better trajectories.

References

- Anthropic. Introducing Claude Opus 5, July 2026. <https://www.anthropic.com/news/claude-opus-5>.
- Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *Advances in Neural Information Processing Systems*, 38, 2025.
- Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. SWE-rebench v2: Language-agnostic SWE task collection at scale. *arXiv preprint arXiv:2602.23866*, 2026.
- BIGBANG team. BigBang: Pursuing open-ended intelligence through self-evolving synthesis of verifiable frontier tasks. Technical report, 2026. <https://github.com/endless-frontier/BigBang-v1>. Preprint. Technical Report.
- Yihan Chen, Benfeng Xu, Xiaorui Wang, Yongdong Zhang, and Zhendong Mao. Training LLM-based agents with synthetic self-reflected trajectories and partial masking. *arXiv preprint arXiv:2505.20023*, 2025.
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, 2024. <https://openai.com/index/introducing-swe-bench-verified/>.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. <https://arxiv.org/abs/2509.16941>.
- Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. Context length alone hurts LLM performance despite perfect retrieval. *arXiv preprint arXiv:2510.05381*, 2025.
- Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. A multi-agent system for automating scientific discovery. *Nature*, pages 1–3, 2026.
- Hao Han, Jin Xie, Xuehao Ma, Weiquan Zhu, Ziyao Zhang, ZhiLiang Long, Hongkai Chen, and Qingwen Ye. SWE-TRACE: Optimizing long-horizon SWE agents through rubric process reward models and heuristic test-time scaling. *arXiv preprint arXiv:2604.14820*, 2026.
- Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. <https://github.com/datacurve-ai/deep-swe>.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. *arXiv preprint arXiv:2504.07164*, 2025.
- Neharika Jali, Anupam Nayak, and Gauri Joshi. Not all turns are equally hard: Adaptive thinking budgets for efficient multi-turn reasoning. *arXiv preprint arXiv:2604.05164*, 2026.
- Yuhua Jiang, Yuwen Xiong, Yufeng Yuan, Chao Xin, Wenyan Xu, Yu Yue, Qianchuan Zhao, and Lin Yan. PAG: Multi-turn reinforced LLM self-correction with policy as generative verifier. *arXiv preprint arXiv:2506.10406*, 2025.
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement. *arXiv preprint arXiv:2607.15524*, 2026a.

- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026b.
- Xiao Liang, Zhong-Zhi Li, Yeyun Gong, Yang Wang, Hengyuan Zhang, Ying Nian Wu, Weizhu Chen, et al. SWS: Self-aware weakness-driven problem synthesis in reinforcement learning for LLM reasoning. *Advances in Neural Information Processing Systems*, 38:56801–56839, 2025.
- Xiao Liang, Zhong-Zhi Li, Yeyun Gong, Yingnian Wu, Zhijiang Guo, Weizhu Chen, et al. Beyond pass@1: Self-play with variational problem synthesis sustains RLVR. In *International Conference on Learning Representations*, volume 2026, pages 79443–79491, 2026a.
- Xiao Liang, Zhong-Zhi Li, Zhenghao Lin, Eric Hanchen Jiang, Hengyuan Zhang, Yelong Shen, Kai-Wei Chang, Ying Nian Wu, Yeyun Gong, and Weizhu Chen. Training LLMs for divide-and-conquer reasoning elevates test-time scalability. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 34402–34427, 2026b.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026a.
- Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents, 2026b. *arXiv preprint arXiv:2605.30621*.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, Zheng Liu, Lei Xiong, Jiahao Wang, Sen Wang, et al. AREX: Towards a recursively self-improving agent for deep research. *arXiv preprint arXiv:2607.21461*, 2026.
- Nikolai Ludwig, Wasi Uddin Ahmad, Somshubra Majumdar, and Boris Ginsburg. From SWE-Zero to SWE-Hero: Execution-free to execution-based fine-tuning for software engineering agents. *arXiv preprint arXiv:2604.01496*, 2026.
- Murong Ma, Tianyu Chen, Yun Lin, Shuai Lu, Qinglin Zhu, Yeyun Gong, Zhiyong Huang, Peng Cheng, Yan Lu, and Jin Song Dong. From patches to trajectories: Privileged process supervision for software-engineering agents. *arXiv preprint arXiv:2605.21996*, 2026.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-Refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition, July 2026. <https://openai.com/index/gpt-5-6/>.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. <https://qwen.ai/blog?id=qwen3.5>.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Yuda Song, Hanlin Zhang, Carson Eisenach, Sham Kakade, Dean Foster, and Udaya Ghai. Mind the gap: Examining the self-improvement capabilities of large language models. In *International Conference on Learning Representations*, volume 2025, pages 39894–39931, 2025.
- Hanlin Wang, Jian Wang, Chak Tou Leong, and Wenjie Li. STeCa: Step-level trajectory calibration for LLM agent learning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11597–11614, 2025.

- Junjie Wang, Zequn Xie, Dan Yang, Jie Feng, Yue Shen, Duolin Sun, Meixiu Long, Yihan Jiao, Zhehao Tan, Jian Wang, et al. WebClipper: Efficient evolution of web agents with graph-based trajectory pruning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21646–21666, 2026a.
- Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. In *COLM 2026 The 2nd Workshop on Lifelong Agents: Learning, Aligning, and Evolving*, 2026b.
- XYZ Agentic Team. AI4AI at scale: A full-pipeline system for enhancing LLM agentic capabilities. Technical report, XYZ AI Lab, 2026. <https://xyz-lab.ai/blogs/ai4ai-at-scale/assets/bounded-exploration-ai4ai-system-optimization.pdf>.
- John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- John Yang, Kilian Lieret, Carlos Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. *Advances in Neural Information Processing Systems*, 38, 2025.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025.
- Lei Zhang, Jiayi Yang, Min Yang, Jian Yang, Mouxiang Chen, Jiajun Zhang, Zeyu Cui, Binyuan Hui, and Junyang Lin. SWE-Flow: Synthesizing software engineering data in a test-driven manner. *arXiv preprint arXiv:2506.09003*, 2025.
- Dewu Zheng, Ruizhe Ye, Yanlin Wang, Yang Ye, Hongyu Zhang, Ensheng Shi, Xilin Liu, Yuchi Ma, Jianxing Yu, and Zibin Zheng. Swe-prime: Fewer trajectories, better performance. *arXiv preprint arXiv:2608.27449*, 2026.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Appendix Contents

A	Hint Generation for Self-Likelihood Guidance	16
B	Trajectory Curation and Distillation Setup	17
B.1	Curating the Training Trajectories	17
B.2	Fine-Tuning and Evaluating the Student	17
C	Additional Ablation Studies	18
C.1	Does a Stronger Judge Help?	18
C.2	Does the Judge Benefit from Hidden Tests?	19
C.3	Conditional Likelihood versus Likelihood Gain	19
D	Details of the Recursive Self-Improvement (RSI) Framework	21
E	Prompts	23
E.1	Trajectory Generation	23
E.2	Self-Judgement with Rubrics	24
E.3	Self-Likelihood Guidance	26

A. Hint Generation for Self-Likelihood Guidance

Motivation. Self-likelihood guidance conditions the candidate log-probability on privileged information (PI) inserted into the context, so the amount of injected context directly shapes the score. The gold patch may be poorly suited for this role because its length varies by orders of magnitude across tasks, and excessively long contextual information can disrupt the structure of trajectory generation, thereby distorting the candidates’ log-probabilities (Du et al., 2025). Over the 766 SWE-bench Verified and Pro (Python) tasks, only 6.7% of patches are shorter than 500 characters, while 21.9% fall between 4,000 and 12,000 characters and a further 8.2% exceed 12,000 (Table 7). The two benchmarks differ sharply: patches on Verified are mostly short, with 71.6% under 1,500 characters, whereas 73.7% of Pro (Python) patches exceed 4,000. The score is therefore confounded with the length of the patch rather than the quality of the candidate, and the shortest patches carry too little signal to separate the candidates. To remove this confound, we compress each gold patch into a prose hint of roughly 1,000 characters, so that every task contributes the same amount of privileged context.

Table 7: Length of the gold patch in characters over the 766 SWE-bench Verified and Pro (Python) tasks.

Patch length (characters)	Verified	Pro (Python)	All
0–500	51	0	51
500–1,500	307	0	307
1,500–4,000	107	70	177
4,000–12,000	30	138	168
Over 12,000	5	58	63
Total	500	266	766

Hint Generation. The hint for each task is produced once, from the problem statement and the gold patch, by GPT-5.6-Sol with the prompt given in Prompt 6. The prompt forbids code, diffs, and any reference to the existence of a patch, and asks for four hints covering where the change belongs, what the defect is, how the fix works, and what to watch out for. The length constraint keeps every hint between 700 and 1,100 characters; the generated hints measure 822–1,043 characters ($\sigma = 37$) over the 766 tasks.

Example. The box below shows the hint for `sympy__sympy-20154`, whose gold patch is 1,558 characters; the hint is 930 characters, the median over the 766 tasks. The patch removes the docstring passage stating that the same dictionary object is returned on every iteration, and replaces two `yield ms` statements with `yield ms.copy()`. The four hints cover the location, the defect, the fix, and the pitfalls, and none contains code: the third describes a shallow snapshot of the dictionary rather than naming the method that produces it.

Hint for `sympy__sympy-20154`

1. Make the change in `sympy/utilities/iterables.py`, within `partitions()`. Inspect both its documentation and every point where the generator emits a partition, including the initial result and the main loop.
2. The generator builds partitions by mutating one internal dictionary and exposes that same mutable object on every iteration. Consumers retaining results, especially through `list(partitions(...))`, therefore see earlier entries change into the final state.
3. Keep the efficient in-place working state, but yield a shallow snapshot of the partition dictionary each time. Apply this consistently whether the iterator returns only a mapping or a size-and-mapping pair.
4. Do not alter partition ordering, multiplicities, limit handling, or the numeric size value; only mapping identity should change. Remember the first yield as well as loop yields, and remove documentation telling callers to copy results themselves.

B. Trajectory Curation and Distillation Setup

This section provides additional details on the trajectory curation settings and student-model distillation experiments described in Section 4.4, including how the training trajectories for the three settings are curated (Appendix B.1) and how the student model is fine-tuned and evaluated (Appendix B.2).

B.1. Curating the Training Trajectories

Task pool. All three experimental groups draw from the same pool of 10,780 runnable Python tasks spanning 797 repositories, sampled from SWE-Smith (Yang et al., 2025) and the SWE-rebench series (Badertdinov et al., 2025, 2026). The pool includes both synthetic tasks and real GitHub pull requests. Each task is accompanied by a container image, a test command, and a gold patch, and the entire pool is decontaminated against all SWE-bench benchmarks in downstream evaluation.

The three experimental groups use the same generator, Qwen3.5-122B-A10B, which produces one trajectory per task under each of the three strategies. We use the same harness and decoding configuration as in the main experiments: mini-SWE-agent with Bash command only, a 250-turn cap, temperature 1.0, top- p 0.95, top- k 10, and at most 4,096 new tokens per turn.

- **Standard:** vanilla turn-by-turn trajectory generation with a single sampled continuation at each turn.
- **Full self-judgement:** self-judgement applied at every turn, with $k = 2$ candidates and $n = 3$ rubric-based judgements per candidate. The judgements assess the soundness of the reasoning and proposed action, as well as alignment with the task goal (Section 4.1).
- **Discovered strategy:** the early-commitment window identified in Section 4.3, where the same self-judgement procedure is applied only from the initial observation through the third file-mutating command.

Because the guidance never enters the policy’s context, the three settings produce trajectories of identical form. We keep every trajectory that exits cleanly by rerunning it if not, solved or not; all three settings retain the same 10,780 tasks, so they are trained and compared on identical problems.

What the curated trajectories look like. Table 8 summarizes the retained trajectories of the three settings. Self-judgement raises the share of solved tasks by 1.6% and shortens trajectories on every statistic, with the mean falling from 64.6 to 57.3 turns. The discovered strategy recovers most of the 1.1% accuracy gain with a smaller reduction in length, consistent with branching only in the early-commitment window. Note that unsolved trajectories are retained in all three settings, and only about half of the tasks are successfully resolved.

One potential limitation of the task pool is that **many tasks are either trivial or beyond the generator’s reach**. These tasks yield similar trajectories under every strategy and thus compress the differences in Table 8. Selecting tasks closer to the generator’s capability boundary is a promising direction for future work.

Table 8: Training trajectories of the three settings on the 10,780 curated tasks (generator: Qwen3.5-122B-A10B). Parentheses give the change against the Standard setting.

Curation strategy	Resolved (%)	Turns		
		Mean	Med.	P90
Standard	46.7	64.6	56	114
Full self-judgement	48.3 (↑1.6)	57.3 (↓7.3)	49 (↓7)	102 (↓12)
Discovered strategy	47.8 (↑1.1)	61.8 (↓2.8)	54 (↓2)	111 (↓3)

B.2. Fine-Tuning and Evaluating the Student

Table 9 lists the fine-tuning and evaluation settings, which are identical for the three students. Each student model is initialized from the Qwen3.5-35B-A3B-Base pretrained checkpoint and fine-tuned for two epochs on

all trajectories retained under the corresponding setting. Trajectories longer than the 131,072-token context are dropped rather than truncated. The loss is computed only on the tokens generated by the model, that is, every assistant turn together with its reasoning trace, while the system prompt, the task description, and all tool outputs are masked out. The three students are then evaluated with the same harness, decoding parameters, and turn cap as the main experiments in Table 1, using the final checkpoint after two epochs.

Setting	Value
<i>Supervised fine-tuning</i>	
Student	Qwen3.5-35B-A3B-Base
Training data	10,780 trajectories
Epochs	2
Global batch size	32 trajectories
Optimizer	AdamW
Learning rate	5×10^{-6} with cosine decay to 5×10^{-7}
Warm-up	3%
Weight decay	0.1
Gradient clipping	1.0
Max sequence length	131,072 tokens
Framework	Megatron-LM, bf16
Hardware	8 AMD MI300X GPUs
<i>Evaluation</i>	
Benchmarks	SWE-bench Verified and SWE-bench Pro (Python)
Harness	mini-SWE-agent, 250-turn cap
Temperature	1.0
Top- p	0.95
Top- k	10
Max new tokens per turn	4,096
Checkpoint	Final checkpoint after two epochs

Table 9: Settings for fine-tuning and evaluating the student.

C. Additional Ablation Studies

C.1. Does a Stronger Judge Help?

Self-judgement uses the policy as its own judge. Table 10 studies whether a stronger judge would select better candidates, replacing the self-judge with GPT-5.6-Terra or GPT-5.6-Sol while keeping the rubric, the gold patch as privileged information, and the policy unchanged. The numbers differ from Table 1 because these runs use a different sampling configuration (temperature 0.6, top- k 20). A stronger judge raises the resolve rate further: on Qwen3.5-35B-A3B from 68.0% under self-judgement to 71.4% with GPT-5.6-Sol, and on Qwen3.5-122B-A10B from 69.6% to 72.6%, while keeping trajectories as concise as the self-judged ones.

These results suggest that self-judgement is sufficient to filter obvious errors and redundant turns, accounting for most of the gains in conciseness, whereas selecting among multiple plausible candidates still benefits from a stronger judge. More broadly, pushing the limits of a model’s problem-solving capabilities through self-improvement may also require enhancing its judgement ability, enabling the policy to more accurately assess the quality of its reasoning and actions and make more effective decisions.

Table 10: Self-judgement with the policy as judge versus a GPT-5.6 judge on SWE-bench Verified. Tokens are the mean number of generated tokens per trajectory. Best value per policy among the judged settings in **bold**; the standard baseline is excluded from the comparison.

Policy	Judge	Resolve rate (%)	Tokens	Turns		
				Mean	Med.	P90
Qwen3.5-35B-A3B	Standard	65.8	19,613	85.0	76	–
	Self	68.0 (↑2.2)	17,782	77.0 (↓8.0)	67 (↓9)	–
	GPT-5.6-Terra	71.2 (↑5.4)	18,054	80.8 (↓4.2)	70 (↓6)	138
	GPT-5.6-Sol	71.4 (↑5.6)	17,755	76.6 (↓8.4)	68 (↓8)	126
Qwen3.5-122B-A10B	Standard	67.0	15,200	72.1	64	–
	Self	69.6 (↑2.6)	16,100	71.0 (↓1.1)	65 (↑1)	121
	GPT-5.6-Sol	72.6 (↑5.6)	15,716	71.0 (↓1.1)	64 (0)	119

C.2. Does the Judge Benefit from Hidden Tests?

In our main experiments (Table 1), self-judgement uses the gold patch as privileged information. Repository-level coding tasks, however, also include test cases that specify the requirements a correct fix must satisfy, providing the judge with additional evidence for assessing whether a candidate better aligns with the task objective. Table 11 examines whether augmenting the judge’s context with these hidden tests improves candidate selection, while keeping the rubric, policy, and sampling configuration unchanged; the additional prompt content is provided in Appendix E.2. Notably, trajectories become more concise in all four settings: on SWE-bench Verified, the mean number of turns decreases from 73.7 to 70.7 for Qwen3.5-35B-A3B and from 68.8 to 65.9 for Qwen3.5-122B-A10B. Adding hidden tests also tends to improve accuracy across models and benchmarks. For example, Qwen3.5-35B-A3B gains 3.3% on SWE-bench Pro, the larger and more challenging benchmark. These results suggest that providing test cases helps the self-judge identify candidates that better satisfy the task requirements, thereby improving both trajectory quality and conciseness.

Table 11: Evaluating whether adding the hidden test cases to the self-judgement context helps candidate selection. Parentheses give the change against the gold patch only.

Model	Privileged information	Resolve rate (%)		Turns – Verified			Turns – Pro		
		Verified	Pro	Mean	Med.	P90	Mean	Med.	P90
Qwen3.5-35B-A3B	Gold patch	70.0	53.2	73.7	67	127	79.3	72	133
	Gold patch + tests	68.8 (↓1.2)	56.5 (↑3.3)	70.7 (↓3.0)	63 (↓4)	123 (↓4)	79.1 (↓0.2)	71 (↓1)	138 (↑5)
Qwen3.5-122B-A10B	Gold patch	71.0	58.5	68.8	61	116	80.2	74	130
	Gold patch + tests	72.8 (↑1.8)	58.5 (0.0)	65.9 (↓2.9)	57 (↓4)	110 (↓6)	78.0 (↓2.2)	72 (↓2)	132 (↑2)

C.3. Conditional Likelihood versus Likelihood Gain

Section 3.3 scores each candidate by its conditional log-likelihood given the privileged information (PI) z in context. Intuitively, the *change in the model’s confidence after inserting the privileged information* may also reflect candidate quality. This gives the *likelihood gain*, which subtracts the unconditioned log-likelihood from Equation (4) and selects the candidate whose likelihood benefits most from z :

$$\Delta\ell(a_t^{(i)}) = \ell(a_t^{(i)}) - \frac{1}{|a_t^{(i)}|} \sum_{u=1}^{|a_t^{(i)}|} \log \pi_\theta(a_{t,u}^{(i)} \mid x, \tau_{<t}, a_{t,<u}^{(i)}), \quad i^* = \arg \max_{i \in [k]} \Delta\ell(a_t^{(i)}), \quad (9)$$

which is the pointwise mutual information between the candidate and z . Table 12 compares these two scoring functions on Qwen3.5-122B-A10B across all four combinations of PI and placement. Although likelihood gain achieves competitive resolve rates in some configurations, it consistently produces longer trajectories. For

example, when the PI is placed before the trajectory, likelihood gain increases the mean trajectory length on SWE-bench Verified by 16 turns rather than reducing it. Conditional likelihood achieves higher or comparable resolve rates in most configurations while consistently producing more concise trajectories; we therefore adopt it in all main experiments.

Table 12: Self-likelihood guidance on Qwen3.5-122B-A10B scored by the conditional likelihood (ℓ , used in the main experiments) or by the likelihood gain over the unconditioned model ($\Delta\ell$, pointwise mutual information), for each choice of PI and placement. Parentheses give the change against Standard.

PI	Placement	Score	Resolve rate (%)		Turns – Verified			Turns – Pro (Python)		
			Verified	Pro (Python)	Mean	Med.	P90	Mean	Med.	P90
Standard			67.0	55.3	72.1	64	122	78.9	74	126
Gold patch	Before	ℓ	67.8 ($\uparrow 0.8$)	56.0 ($\uparrow 0.7$)	71.5 ($\downarrow 0.6$)	62 ($\downarrow 2$)	129 ($\uparrow 7$)	66.0 ($\downarrow 12.9$)	61 ($\downarrow 13$)	110 ($\downarrow 16$)
		$\Delta\ell$	70.6 ($\uparrow 3.6$)	57.9 ($\uparrow 2.6$)	88.2 ($\uparrow 16.1$)	78 ($\uparrow 14$)	166 ($\uparrow 44$)	81.4 ($\uparrow 2.5$)	70 ($\downarrow 4$)	146 ($\uparrow 20$)
Hint	Before	ℓ	69.8 ($\uparrow 2.8$)	60.5 ($\uparrow 5.2$)	69.5 ($\downarrow 2.6$)	62 ($\downarrow 2$)	118 ($\downarrow 4$)	65.3 ($\downarrow 13.6$)	60 ($\downarrow 14$)	114 ($\downarrow 12$)
		$\Delta\ell$	69.8 ($\uparrow 2.8$)	60.9 ($\uparrow 5.6$)	88.2 ($\uparrow 16.1$)	80 ($\uparrow 16$)	151 ($\uparrow 29$)	92.1 ($\uparrow 13.2$)	83 ($\uparrow 9$)	154 ($\uparrow 28$)
Gold patch	After	ℓ	69.4 ($\uparrow 2.4$)	57.5 ($\uparrow 2.2$)	64.6 ($\downarrow 7.5$)	60 ($\downarrow 4$)	108 ($\downarrow 14$)	62.8 ($\downarrow 16.1$)	58 ($\downarrow 16$)	106 ($\downarrow 20$)
		$\Delta\ell$	68.6 ($\uparrow 1.6$)	54.5 ($\downarrow 0.8$)	67.1 ($\downarrow 5.0$)	62 ($\downarrow 2$)	111 ($\downarrow 11$)	62.5 ($\downarrow 16.4$)	58 ($\downarrow 16$)	102 ($\downarrow 24$)
Hint	After	ℓ	68.8 ($\uparrow 1.8$)	59.4 ($\uparrow 4.1$)	65.3 ($\downarrow 6.8$)	59 ($\downarrow 5$)	110 ($\downarrow 12$)	66.1 ($\downarrow 12.8$)	61 ($\downarrow 13$)	108 ($\downarrow 18$)
		$\Delta\ell$	69.8 ($\uparrow 2.8$)	55.6 ($\uparrow 0.3$)	67.5 ($\downarrow 4.6$)	64 (0)	112 ($\downarrow 10$)	65.2 ($\downarrow 13.7$)	62 ($\downarrow 12$)	106 ($\downarrow 20$)

D. Details of the Recursive Self-Improvement (RSI) Framework

This section documents all 15 cycles of the RSI framework for discovering more efficient turn-level self-improvement strategies described in Section 3.4, including the strategy proposed in each cycle (Table 14) and its performance on the validation split (Table 15).

Setup. Table 13 lists the settings that are specific to the framework. Each cycle evaluates exactly one proposed strategy against the incumbent on the full validation split, so a cycle costs one complete generation pass over 192 tasks. A strategy is described by a *branching rule*, which decides at which turns to draw $k > 1$ candidates, and a *selector*, which is either self-judgement (SJ, Section 3.2) or self-likelihood (SL, Section 3.3). Both signals are used solely for candidate selection in every cycle: neither the gold patch provided to the rubric judge, the hint used for likelihood scoring, nor any resulting feedback is ever exposed to the policy’s generation context.

What the record shows. Most cycles are rejected for a consistent reason: a strategy that branches rarely leaves accuracy inside the noise band while adding branching cost. Cycles 3–5 illustrate this failure mode, each gaining six solved tasks, one below the acceptance threshold $\delta = 7$, at 3–19% branching and no reduction in trajectory length. The strategy accepted at cycle 6 wins on conciseness: it branches on 36.6% of turns, yet its retained trajectories carry $0.87\times$ the tokens and $0.93\times$ the turns of the incumbent at unchanged accuracy, because **the early-commitment window removes wandering turns rather than adding candidates late**.

Every proposal is screened for whether it alters the policy’s generation context, and any proposal that does so is rejected regardless of its performance gain. Cycle 14 provides a representative example: although it achieves the largest accuracy improvement, we exclude it as inadmissible because it violates our constraint that guided trajectories must remain indistinguishable in form from standard rollouts and thus valid as training data. Specifically, it appends a judge-generated note to the agent’s next observation, causing PI to enter the policy’s generation context. This violates our invariant that every arm must expose the policy to exactly the same inputs as an unguided rollout. Because the judge has access to the gold patch, this effectively introduces answer leakage: the resulting trajectories show the agent reproducing the judge’s note before acting on it, rather than improving through reasoning-based candidate selection. We thus carry the Cycle 6 strategy, rather than the Cycle 14, forward to the OOD generalization evaluation in Section 4.3 and data curation in Section 4.4.

Table 13: Settings of the RSI framework for self-improvement strategy optimization.

Setting	Value
Proposer	Either Claude Opus 5 or GPT-5.6-Sol
Recorder	Either Claude Opus 5 or GPT-5.6-Sol
Executor	Qwen3.5-122B-A10B
Initial strategy pool	Standard generation, every-turn SJ, and every-turn SL
Validation split	192 tasks SWE-bench Multilingual
Samples	One trajectory per task per cycle
Temperature	1.0
Top- k	10
Max new tokens per turn	4,096
Turn cap	250
Initial Incumbent	Standard decoding
Acceptance (accuracy)	$\Delta_t \geq \delta = +7$ solved tasks
Acceptance (efficiency)	Trajectory tokens or turns $\leq 0.9\times$ the incumbent with $\Delta_t \geq -2$
Disqualification	Any proposal that alters the policy’s generation context

Table 14: The 15 proposed strategies. **Method** refers to the turn-level self-improvement strategy: **SJ** scores candidates with the PI-aware rubric, while **SL** scores them by their likelihood under the PI. k is the number of candidates drawn at a branching turn; every other turn draws one.

Cyc.	Method	k	Branching rule and selector
1	SJ	4	Branch when the last command exited non-zero or its output was truncated.
2	SL	6	Branch when the last four commands fall in one family (test / modify / first-executable) and the last exit code is 0.
3	SL	6	Branch when the last exit code is non-zero and the previous two commands are in the same family.
4	SJ	2	After the first edit, branch at the turn that follows an executed verification command.
5	SJ	2	Branch at every turn before the first workspace mutation.
6	SJ	2	<i>Early-commitment window</i> : branch until the third mutation-like command has executed, then follow a single path.
7	SL	6	Branch at the turn after the agent successfully reviews its own changes (a diff, a review command, or unified-diff output).
8	SJ	2	Same trigger as cycle 7, scored by the rubric: the continue-versus-submit checkpoint.
9	SJ	2	Cycle-6 trigger; a challenger replaces the first draft only if all three of its samples exceed all three of the draft's.
10	SL	8	Branch at the first turn after a passed test, behavioral probe, or build check; consecutive checkpoints suppressed.
11	SJ	2	Cycle-6 trigger with two judge samples per candidate instead of three.
12	SJ	2	Cycle-6 trigger; the rubric is replaced by a single bottleneck-scoring prompt.
13	SL	6	Cycle-3 trigger, ranked by the likelihood gain (plain minus privileged NLL) instead of the privileged likelihood.
14	SJ	2	After an edit and a successful diff review, keep the first draft but let the judge name the highest-impact omission and queue that note into the next observation.
15	SJ	4	After the workspace has been edited, branch when the last probe, build, or test failed; duplicate commands are collapsed before scoring.

Table 15: Validation results of the 15 cycles on the 192-task split. Δ_t is the change in solved tasks against the incumbent of that cycle. **Branch** is the number of candidates that the strategy drew at branching turns. **Tok.** is retained-trajectory tokens relative to the incumbent.

Cyc.	Solved		Branch (%)	Turns			Tok.	Decision
	/192	Δ_t		Mean	Med.	P90		
–	99	–	0.0	72.5	64	119	1.00	incumbent
1	94	–5	8.3	60.1	56	104	0.78	reject
2	99	0	13.3	72.2	66	114	1.02	reject
3	105	+6	3.4	72.4	64	121	1.02	reject
4	105	+6	12.3	70.2	63	112	1.03	reject
5	105	+6	19.4	66.8	60	119	1.03	reject
6	101	+2	36.6	62.1	59	104	0.87	accept (cost)
7	105	+4	4.8	74.8	67	119	1.21	reject
8	106	+5	4.7	75.2	64	127	1.24	reject
9	102	+1	36.6	70.9	62	116	1.14	reject
10	98	–3	4.7	73.0	62	121	1.21	reject
11	102	+1	35.6	73.5	65	128	1.29	reject
12	97	–4	37.5	68.2	63	107	1.05	reject
13	101	0	3.5	73.7	63	121	1.26	reject
14	119	+18	2.7	97.6	84	177	1.72	reject (context)
15	103	+2	6.1	71.6	59	112	1.11	reject

E. Prompts

This section lists, verbatim, every prompt used in the experiments of Section 4. Placeholders in double braces (`{{task}}`, `{{reference_patch}}`, ...) are substituted textually at run time; em-dashes in the originals are rendered as `-` here. Nothing is added to the policy’s context by any of the guidance methods: Sections E.2 and E.3 are seen only by the scoring pass.

E.1. Trajectory Generation

The policy runs under the mini-SWE-agent `swebench` configuration, reproduced here from the harness. Every run sends the two messages below together: a one-line system message, followed by a first user message that wraps the issue text and gives the bash-only protocol, the recommended workflow and the submission procedure. Environment results and format-error notices follow the harness defaults and are omitted here.

Prompt 1: Prompts for trajectory generation: the system message followed by the first user message.

```
=== System message ===
You are a helpful assistant that can interact with a computer shell to solve programming tasks.

=== First user message ===
<pr_description>
Consider the following PR description:
{{task}}
</pr_description>

<instructions>
# Task Instructions

## Overview

You're a software engineer interacting continuously with a computer by submitting commands.
You'll be helping implement necessary changes to meet requirements in the PR description.
Your task is specifically to make changes to non-test files in the current directory in order to fix the issue
described in the PR description in a way that is general and consistent with the codebase.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result,
then think and issue your next command(s).</important>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

## Important Boundaries

- MODIFY: Regular source code files in /testbed (this is the working directory for all your subsequent commands)
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

## Recommended Workflow

1. Analyze the codebase by finding and reading relevant files
2. Create a script to reproduce the issue
3. Edit the source code to resolve the issue
4. Verify your fix works by running your script again
5. Test edge cases to ensure your fix is robust

## Command Execution Rules

You are operating in an environment where

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

**CRITICAL REQUIREMENTS:**

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. You can make MULTIPLE tool calls in a single response
when the commands are independent (e.g., searching multiple files, reading different parts of the codebase).
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.
```

```

- However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load
environment variables from files

Example of a CORRECT response:
<example_response>
I need to understand the Builder-related code. Let me find relevant files and check the project structure.

[Makes multiple bash tool calls: {"command": "ls -la"}, {"command": "find src -name '*.java' | grep -i builder"},
{"command": "cat README.md | head -50"}]
</example_response>

## Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input
- You can use bash commands or invoke any tool that is available in the environment
- You can also create new tools or scripts to help you with the task
- If a tool isn't available, you can also install it

## Submission

When you've completed your work, you MUST submit your changes as a git patch.
Follow these steps IN ORDER, with SEPARATE commands:

Step 1: Create the patch file
Run `git diff -- path/to/file1 path/to/file2 > patch.txt` listing only the source files you modified.
Do NOT commit your changes.

<IMPORTANT>
The patch must only contain changes to the specific source files you modified to fix the issue.
Do not submit file creations or changes to any of the following files:

- test and reproduction files
- helper scripts, tests, or tools that you created
- installation, build, packaging, configuration, or setup scripts unless they are directly part of the issue you
were fixing (you can assume that the environment is already set up for your client)
- binary or compiled files
</IMPORTANT>

Step 2: Verify your patch
Inspect patch.txt to confirm it only contains your intended changes and headers show `--- a/` and `+++ b/` paths.

Step 3: Submit (EXACT command required)
You MUST use this EXACT command to submit:

```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat patch.txt
```

If the command fails (nonzero exit status), it will not submit.

<CRITICAL>
- Creating/viewing the patch and submitting it MUST be separate commands (not combined with &&).
- If you modify patch.txt after verifying, you SHOULD verify again before submitting.
- You CANNOT continue working (reading, editing, testing) in any way on this task after submitting.
</CRITICAL>
</instructions>

```

E.2. Self-Judgement with Rubrics

At a branching turn each candidate is scored in a separate request. The request has a fixed system prompt (below; {formula} is filled with the weighted sum) and a user message that serializes the issue, the trajectory so far with the agent's internal reasoning removed, and the candidate's analysis and tool call. Three samples are drawn per candidate and the weighted totals are averaged; selection uses our own weighted sum of the item scores, not the judge-reported total.

Rubric weights. Without privileged information (*judge w/o PI*, Table 5): R1 0.15, R2 0.15, R3 0.10, A1 0.15, A2 0.20, A3 0.15, A4 0.10. With privileged information (*judge w/ PI*, all self-guided rows of Table 1): the same items scaled by 0.70 – R1 0.105, R2 0.105, R3 0.07, A1 0.105, A2 0.14, A3 0.105, A4 0.07 – plus G1 0.30.

Prompt 2: Judge system prompt with the privileged information (seven rubric items plus G1).

You are an expert software-engineering supervisor. A coding agent is solving a real GitHub issue in a repository sandbox. You will see the trajectory so far (issue description, the agent's previous tool calls, and the environment's results -- the agent's internal reasoning has been removed), followed by ONE candidate next turn proposed by the agent. The candidate contains the agent's ANALYSIS (its reasoning for this turn) and its TOOL_CALL (the concrete action it proposes). The candidate has NOT been executed yet -- judge it on its merits given the evidence so far. Score on an absolute scale: other candidates are evaluated in separate requests, so anchor your scores to the rubric definitions, not to comparisons.

You are ALSO given the REFERENCE SOLUTION: the patch that actually resolved this issue upstream. The agent cannot see it and must not be expected to reproduce it verbatim. Use it only as ground truth about WHERE the real defect lives and WHAT ultimately had to change.

Score the candidate on every rubric from 1 (very poor) to 10 (excellent):

Reasoning rubrics (about ANALYSIS):

- R1 Groundedness: the analysis is consistent with evidence in the trajectory (files actually read, real error messages, real test output); it does not hallucinate files, APIs, or results.
- R2 Diagnostic insight: the reasoning meaningfully narrows down the root cause or reduces uncertainty; it forms or updates concrete hypotheses rather than vaguely restating the problem.
- R3 Plan coherence: the stated plan follows logically from the analysis and advances the overall task; it is not circular or redundant with prior turns.

Action rubrics (about TOOL_CALL):

- A1 Analysis-action alignment: the tool call actually implements what the analysis proposes.
- A2 Correctness & specificity: tool name, arguments, file paths, and commands are well-formed and plausibly valid for this repository; edits are precise and minimal rather than broad rewrites.
- A3 Expected progress / information gain: executing this action is likely to move the trajectory closer to a correct fix, or to yield high-value new information; it does not repeat actions that already failed.
- A4 Efficiency & safety: avoids destructive, wasteful, or unnecessarily broad operations (mass rewrites, irrelevant exploration, redundant re-reads).

Goal rubric (about convergence on the REFERENCE SOLUTION):

- G1 Goal alignment: does this turn move the trajectory toward the reference solution -- the files, functions, and root cause it touches? Investigating the right module, reproducing the right failure, or editing the right code scores high; wandering into unrelated code, or editing the wrong place, scores low. Judge CONVERGENCE, not textual similarity. A different but equally correct implementation of the same fix scores HIGH. An action that merely quotes or restates the reference without doing the work scores LOW. Early exploratory turns that legitimately narrow toward the right area score in the middle -- do not punish a turn for not being the final edit.

Weighted total = {formula}

Respond with STRICT JSON only, no markdown, in this schema:

```
{
  "R1": _, "R2": _, "R3": _, "A1": _, "A2": _, "A3": _, "A4": _, "G1": _,
  "total": _,
  "justification": "<2-3 sentences>"
}
```

Judge user message (layout). The trajectory is serialized turn by turn; tool outputs are head/tail-elided per block and the oldest blocks are dropped first when the 80,000-character budget is exceeded. The reference-solution section is appended only under *w/ PI*, after the trajectory budget has been spent, so both settings see the same trajectory. The gold patch in that section is itself truncated to 16,000 characters, which affects only the small fraction of tasks whose patch exceeds that length.

Prompt 3: Judge user message layout.

```
## Issue

{issue description, taken from the <pr_description> block of the task prompt}

## Trajectory so far

### Turn 0 -- agent tool call
bash: {command}

### Turn 0 -- environment result
{tool output, head/tail-elided to the per-block budget}

### Turn 1 -- agent tool call
...
[... N earlier blocks elided to fit the context budget ...] (oldest blocks dropped first)

## Reference solution (ground truth -- the agent cannot see this) <-- only under "w/ PI"
```

```

This patch resolved the issue upstream. Use it to judge G1: whether the candidate turn converges on the right
files, functions and root cause.
An equally correct but differently written fix is still fully correct.

```diff
{gold patch, truncated to 16,000 characters}
```

## Candidate next turn

ANALYSIS:
{the candidate's reasoning for this turn}

TOOL_CALL:
bash: {the candidate's command}

```

Hidden tests. The ablation of Appendix C.2 appends one further block to the judge user message, immediately after the reference solution and only when a gold patch is present. The system prompt, the rubric items and their weights are unchanged, so the tests are folded into the existing G1 item rather than scored separately. The block lists the task’s FAIL_TO_PASS and PASS_TO_PASS test identifiers together with the source of the failing tests, which is extracted from the task’s own container image. At most 80 failing identifiers, 15 passing identifiers and 12,000 characters of test source (3,000 per test) are kept; over the 731 SWE-bench Pro tasks the rendered block has a median length of 3,652 characters and a 90th percentile of 12,028. Test source is available for 654 of those tasks, and the remaining 77 list identifiers only. Unlike the main judge prompt, the gold patch is not truncated in this setting.

Prompt 4: Hidden-test block, appended to the judge user message after the reference solution.

```

## Hidden tests (ground truth -- the agent cannot see these)

These tests currently FAIL and must PASS once the issue is correctly fixed
(FAIL_TO_PASS); the listed PASS_TO_PASS tests must keep passing. Together
with the reference solution they define what a correct fix has to achieve.
Use them for G1: a turn that moves toward making these tests pass -- the
behaviours, code paths and edge cases they exercise -- converges; a turn
that would break the PASS_TO_PASS tests or ignores what the failing tests
check does not. Judge CONVERGENCE, not whether the agent runs these exact
tests.

### FAIL_TO_PASS ({N} tests must go from failing to passing)
- {test identifier}
- ... and {k} more

### Source of the failing tests ({k} shown, {m} more omitted for length)
```
{test source, truncated to 3,000 characters per test}
```

### PASS_TO_PASS ({M} existing tests must keep passing)
- {test identifier}
- ... and {k} more

```

E.3. Self-Likelihood Guidance

Each candidate is scored by one forward pass in which a privileged block is inserted into the context; the candidate with the lowest mean NLL under that context is executed. The block is never part of the generation context. Three variants were evaluated (Table 6); the main results use *hint*, *before*.

Prompt 5: Privileged block inserted before the trajectory, holding the hint (default configuration).

```

Here are some hints for this issue:
=== Hints Begin ===
{{reference_patch}}
=== Hints End ===

After reading the hints above, make sure you truly understand what they point to and why -- do not copy or
paraphrase them, and do not mention them. Work through the task with your own reasoning, as you otherwise would.

```

Hint generation prompt. Prompt 6 generates the hint for each task from the problem statement and the gold patch (Appendix A).

Prompt 6: Hint generation prompt.

```
You are helping build a hint set for a software-engineering benchmark.

Below are an issue report from a real repository and the patch that was actually accepted to fix it. Write hints that would help an engineer who can see the issue but NOT the patch.

=== Issue Begin ===
{{problem_statement}}
=== Issue End ===

=== Accepted Patch Begin ===
{{gold_patch}}
=== Accepted Patch End ===

Write exactly 4 hints, as a numbered list, covering in this order:
1. WHERE the change belongs -- name the file(s) and the function/class/section.
2. WHAT the defect actually is -- the underlying cause, in your own words.
3. HOW the fix works -- the shape of the change, described in prose.
4. WHAT to watch out for -- a trap, an edge case, or something the patch deliberately preserves.

Hard rules:
- Do NOT paste code, diffs, patch syntax, or literal lines from the patch. Describe changes in words. Identifiers and file paths are allowed.
- Do NOT mention that a patch exists, or refer to "the patch"/"the fix that was applied". Write as advice to someone about to solve the issue.
- Each hint is 1-2 sentences and at most 240 characters.
- The whole list must be between 700 and 1100 characters, and should land close to 900. Being under the cap matters more than saying everything: drop the least useful detail rather than run long.
- Output the numbered list only: no preamble, no closing remark, no headers.
```